Python for Data Science

Session 1 — Getting the Data In

Andrea Guido

PSB Paris School of Business — PGE 5

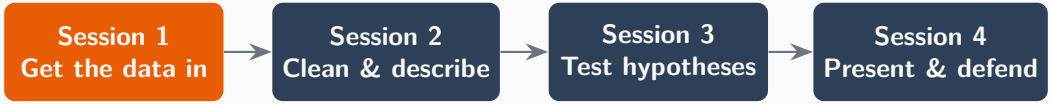
30,000 credit card holders at a Taiwanese bank, 2005

22%

failed to pay the following month

Could the bank have seen it coming? That is our question for the next four sessions.

Every session gives you the tools to answer the next part of the question



Command of the code

Can you explain, modify, and justify every line you wrote?

Critical thinking

Does your method answer the question?
Do you know the limits of your answer?

The data: one row per client, six months of card history

- 30,000 clients of one bank in Taiwan
- April–September 2005, amounts in NT$
- Credit limit, sex, education, marital status, age
- Each month: repayment status, bill, amount paid
- Outcome: did the client default in October?

Source

UCI Machine Learning Repository, dataset #350.

Yeh & Lien (2009), *Expert Systems with Applications* 36(2).

Real data, released as is. It has problems. Finding them is part of the job.

Three research questions, one per test you will learn

RQ1

Do default rates differ across **education levels**?

Two categorical variables

RQ2

Do clients who default have lower **credit limits**?

Comparing two groups

RQ3

Does **recent repayment behaviour** predict default, beyond limit and demographics?

Many variables at once

Why it matters: a bank sets limits and prices credit using exactly these kinds of signals.

Four sessions, each with work before class and an output in class

	Topic	Before class	In class (feeds your presentation)
S1	Getting the data in	install Python (setup guide)	import the data; first summary; variables for each RQ
S2	Cleaning and describing	20-min video: cleaning in Pandas	cleaned dataset, cleaning log, statistics and plots
S3	Testing hypotheses	20-min video: tests and regression	one test for your RQ, interpreted
S4	Presenting and defending	**submit your slides**	group presentation and individual questions

3h30 per session, 14 hours in total. Every session builds on the notebook of the previous one.

Your grade: half group presentation, half final exam

50%

Group presentation

- Groups of **at most 3** students
- One research question per group
- 10 minutes, then 5 minutes of questions
- Slides due **before Session 4**

50%

Final exam

Individual. It tests the same skills you practise in every session: reading, writing and interpreting code on real data.

Both parts are graded on the same two criteria: **command of the code** and **critical thinking**.

Today you leave with a loaded dataset and a map from questions to columns

1. **Python essentials**: the minimum you need, taught on our data
2. **Reading object syntax**: what `df.method()` means
3. **Importing data**: CSV, Excel, and the ways it goes wrong
4. **First inspection**: five commands that show the shape of any dataset
5. **From questions to variables**: which columns matter, and what they really measure

Part 1: Python essentials

Only what we need, all of it applied to credit cards

A notebook runs code one cell at a time, top to bottom

```
2 + 3
```

```
5
```

Shift + Enter runs a cell. The last line's value is printed below it.

Setup

Use Jupyter (Anaconda or VS Code) or Google Colab.

Open `session1.ipynb` and run the first cell now.

Why it matters: if you run cells out of order, the notebook's state no longer matches what you read. When in doubt: *Restart & Run All*.

A variable is a name attached to a value

```
limit = 20000        # credit limit in NT$
age = 24
limit_eur = limit / 35   # rough exchange rate
limit_eur
```

```
571.4285714285714
```

- `=` means "store", not "is equal to"
- Names are case-sensitive: `Limit` and `limit` are two different variables
- Choose names that say what they hold: `limit_eur`, not `x2`

Every value has a type, and the type decides what you can do with it

```
type(20000)       # int
type(0.2212)      # float
type("female")    # str
type(True)        # bool
```

```
"20000" + 5
```

```
TypeError: can only concatenate
str (not "int") to str
```

Why it matters: a number stored as text looks right on screen but cannot be averaged. You will meet this today.

A list holds values in order, and counting starts at zero

```
bills = [3913, 3102, 689, 0, 0, 0]   # Sep, Aug, Jul, Jun, May, Apr
bills[0]       # 3913 -> first element (September)
bills[-1]      # 0    -> last element (April)
bills[0:3]     # [3913, 3102, 689] -> elements 0, 1, 2
len(bills)     # 6
```

- `bills[0:3]` includes the start and **excludes** the end
- Keep this in mind: in our file, September's repayment column is called `PAY_0`

A dictionary maps names to values, and that is what a row of data is

```
client = {"ID": 1, "LIMIT_BAL": 20000, "SEX": 2,
          "AGE": 24, "default": 1}
client["LIMIT_BAL"]          # 20000
client["AGE"] = 25           # change a value
client.keys()                # all the column names
```

Key = column name
always a unique label

Value = the entry
any type: number, text, even a list

Why it matters: Pandas selects columns with the same square brackets: `df["LIMIT_BAL"]`.

A loop repeats an action for each element; `if` decides which action

```
pay_status = [2, 2, -1, -1, -2, -2]   # months late, Sep -> Apr

late_months = 0
for status in pay_status:
    if status >= 1:
        late_months = late_months + 1

late_months
```

```
2
```

Indentation is not decoration: it tells Python which lines belong to the loop and which to the `if`.

A function packages a calculation so you write it once and reuse it

```
def utilisation(bill, limit):
    """Share of the credit limit used by the bill."""
    return bill / limit

utilisation(3913, 20000)    # 0.19565
utilisation(150000, 120000) # 1.25 -> over the limit?
```

- `def` names the function; the arguments go in brackets
- `return` sends the result back to whoever called it
- The second call already raises a data question. Keep it for Session 2.

In practice, Pandas does the loops for you

Plain Python

You loop over 30,000 rows yourself, one client at a time.



Pandas

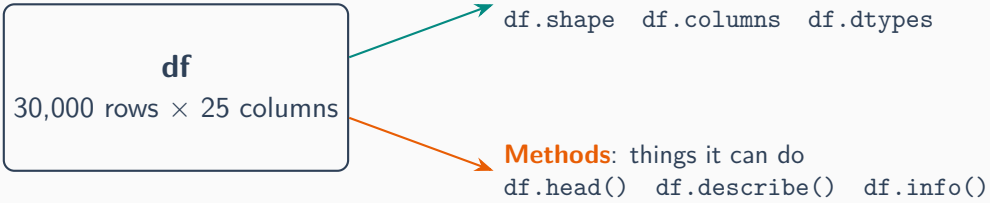
One line acts on a whole column: `df["AGE"].mean()`

Why it matters: you still need loops, lists and functions to *read* Pandas code and to automate repeated steps (e.g., six monthly columns).

Part 2: Reading object syntax

You do not need to write classes to understand `df.method()`

A DataFrame is an object: data plus the tools that work on it



Rule of thumb: **parentheses** mean "do something"; **no parentheses** mean "tell me something".

Read a line of Pandas from left to right, one dot at a time

```
df["LIMIT_BAL"].describe().round(0)
```

`df`	start from the whole table
`["LIMIT_BAL"]`	keep one column (a *Series*)
`.describe()`	compute summary statistics of that column
`.round(0)`	round the result to whole numbers

Each step returns a new object, and the next dot acts on *that* object.

Arguments change what a method does; the documentation lists them

```
df.head()           # first 5 rows (default)
df.head(10)         # first 10 rows
df.sort_values("LIMIT_BAL", ascending=False).head(3)
```

Getting help

`df.head?` in Jupyter, **Shift+Tab** inside the brackets, or the official Pandas documentation.

Reading errors

Start from the **last line** of the error message: it names the problem. The lines above show where it happened.

Part 3: Importing data

Most failures happen before the analysis starts

One import line, one function per file format

```
import pandas as pd

df = pd.read_csv("data/raw/clients.csv")          # text, comma-separated
df = pd.read_excel("data/raw/clients.xlsx")       # Excel workbook
```

- `import pandas as pd`: load the library once, call it `pd` after that
- Use **relative paths** from the project folder, with forward slashes
- Never edit the raw file by hand. Every change happens in code, so it can be checked and repeated.

European CSV files often need three extra arguments

```
df = pd.read_csv("export_bank.csv",
                 sep=";",            # columns split by semicolons
                 decimal=",",        # 1234,56 means 1234.56
                 encoding="latin-1") # accents from older systems
```

Symptom	Likely fix
One single column containing `a;b;c`	`sep=";"`
Amounts load as text: `"1234,56"`	`decimal=","`
`UnicodeDecodeError`, or garbled accents	`encoding="latin-1"`

Failure 1: an old Excel format needs an extra package

```
df = pd.read_excel("data/raw/default of credit card clients.xls")
```

```
ImportError: Missing optional dependency 'xlrd'.
Install xlrd >= 2.0.1 for xls Excel support.
```

Fix

Run `pip install xlrd` (or `!pip install xlrd` in a cell), then restart the kernel.

Lesson

The last line of the error told you exactly what to do. Read it before searching online.

Failure 2: the file loads, but every column is wrong

```
df = pd.read_excel("data/raw/default of credit card clients.xls")
df.head(2)
```

```
  Unnamed: 0         X1    X2         X3        X4   X5 ...
0         ID  LIMIT_BAL   SEX  EDUCATION  MARRIAGE  AGE ...
1          1      20000     2          2         1   24 ...
```

- No error message, and yet **no column is numeric**: the real names sit in row 0 as text
- `df["X1"].mean()` fails, because the column mixes text and numbers
- The file has an extra title row above the real header

One argument fixes it: tell Pandas where the header is

```
path = "data/raw/default of credit card clients.xls"
df = pd.read_excel(path, header=1)   # header = row 1, counting from 0
df.shape
```

```
(30000, 25)
```

Why it matters: a silent failure is worse than an error. Always look at the data right after importing it.

Failure 3: Python cannot find the file

```
FileNotFoundError: [Errno 2] No such file or directory:
'default of credit card clients.xls'
```

Check where you are

```
import os
os.getcwd()    # where am I?
os.listdir()   # what is here?
```

Usual causes

The notebook runs from another folder; a typo in the name; Windows backslashes (\) in the path. Use forward slashes.

Part 4: First inspection

Five commands that show the shape of any dataset

`.shape` and `.head()`: how big is it, and what is a row?

```
df.shape                    # (30000, 25)
df.head(3)
```

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	...
0	1	20000	2	2	1	24	2	2	...
1	2	120000	2	2	2	26	-1	2	...
2	3	90000	2	2	2	34	0	0	...

- One row = one client; 30,000 clients and 25 columns
- Look closely at the column names: `PAY_0`, then `PAY_2`. Where is `PAY_1`?

`.info()` and `.dtypes`: the computer's type is not the variable's meaning

```
df.dtypes.value_counts()
```

```
int64    25
```

Truly numeric

`LIMIT_BAL`, `AGE`, `BILL_AMT*`, `PAY_AMT*`: averages make sense.

Categories stored as numbers

`SEX`, `EDUCATION`, `MARRIAGE`: an "average education of 1.85" means nothing.

Why it matters: the method you choose in Session 3 depends on the variable's meaning, not on its dtype.

`.describe()` gives the summary statistics, and they already raise questions

```
df[["LIMIT_BAL", "AGE", "BILL_AMT1", "EDUCATION"]].describe()
```

	LIMIT_BAL	AGE	BILL_AMT1	EDUCATION
mean	167484.3	35.5	51223.3	1.85
min	10000.0	21.0	-165580.0	0.00
max	1000000.0	79.0	964511.0	6.00

- A **negative** bill of −165,580 NT$: an error, or a refund?
- EDUCATION runs from 0 to 6, but the codebook defines only 1–4

`.value_counts()` shows codes the documentation never mentions

```
df["EDUCATION"].value_counts().sort_index()
```

Code	Clients	Codebook says
0	14	nothing
1	10,585	graduate school
2	14,030	university
3	4,917	high school
4	123	others
5, 6	331	nothing

345 clients have an undocumented education code. RQ1 depends on this column. *What would you do?* (Session 2.)

Write down every oddity now; decide what to do about it in Session 2

What we found	Where
Header on the second row	file structure
`PAY_0` followed by `PAY_2`: no `PAY_1`	column names
Categories stored as integers	`SEX`, `EDUCATION`, `MARRIAGE`
Undocumented codes 0, 5, 6	`EDUCATION` (and 0 in `MARRIAGE`)
Negative bills	`BILL_AMT1`: 590 clients

Why it matters: an oddity you have not written down is an oddity you will forget to justify at the oral exam.

Part 5: From questions to variables

Which columns matter, and what do they actually measure?

Each research question needs an outcome, a main variable, and context

	Outcome	Main variable	Context
RQ1	default next month	`EDUCATION`	—
RQ2	`LIMIT_BAL`	default next month	—
RQ3	default next month	`PAY_0 ... PAY_6`	`LIMIT_BAL, AGE, SEX, ...`

Note that in RQ2 the roles flip: we compare the *limit* between defaulters and non-defaulters. Most of the 25 columns are not needed for any single question.

“Default” here means one missed payment in one month

What the column says

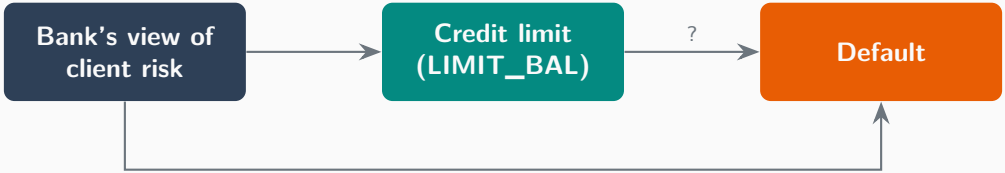
`default payment next month` $= 1$ if the client did not pay in October 2005.

What it does not say

Whether the debt was later repaid. How much was missed. Whether this is “default” in the legal or accounting sense.

Why it matters: your conclusions can only be as precise as your outcome variable. Say “missed next payment”, not “went bankrupt”.

The credit limit is a decision by the bank, not a trait of the client



If defaulters have lower limits (RQ2), is that because low limits cause default, or because the bank *expected* them to default?

Why it matters: this is the difference between a correlation and a cause. We come back to it in Session 3.

The repayment codes −2 and 0 are used but never defined

PAY_0 value	Clients	Codebook says
−2	2,759	nothing
−1	5,686	paid duly
0	14,737	nothing
1 to 8	6,818	months of payment delay

Half the sample has the value 0, which the codebook never defines. A common reading is "paid the minimum, rest carried over", but that is an *assumption*. RQ3 relies on this variable, so you must state which reading you use.

In-class exercise (feeds your group presentation)

1. **Import** the raw file into a DataFrame. Show that the import is correct (shape, dtypes, first rows).
2. **Summarise** it: `.describe()` on the numeric variables, `.value_counts()` on the categorical ones, and the share of clients who default.
3. **List** every oddity you find, with the line of code that reveals it.
4. **For each RQ**, write 3–5 sentences: which columns you need, what each one measures, and one limit of that measurement.

Work in `session1.ipynb`. Submit the notebook with every cell run from top to bottom.

Before you analyse a number,
find out what it actually measures.

Next session: cleaning the data and describing it, and documenting every decision.